HANZO FORMAL VERIFICATION

Proving Infrastructure Correctness with Lean 4

Zach Kelling

Hanzo AI

March 2026

Abstract

We present the formal verification suite for Hanzo infrastructure, comprising over 12,000 lines of Lean 4 proofs covering critical correctness and security properties of five core systems: (i) the agent framework, proving safe delegation, permission bound propagation, and credential chain validity, (ii) the API gateway, proving request routing correctness, rate limiter fairness, and authentication chain soundness, (iii) KMS secrets management, proving envelope encryption correctness, key rotation safety (no secret exposure during rotation), and access control policy consistency, (iv) the compute scheduler, proving resource allocation fairness, deadlock freedom, and SLA compliance bounds, and (v) the platform orchestrator, proving deployment state machine correctness, rollback safety, and health check liveness. Each proof is mechanically verified by the Lean 4 kernel—no axioms beyond the Lean standard library are assumed. We describe the verification methodology, present the key theorems and their proof structures, and report on the 8 bugs discovered during formalization that were present in the production codebase but undetected by testing.

1 Introduction

Testing demonstrates the presence of bugs; formal verification demonstrates their absence. Hanzo infrastructure handles authentication for 2,800+ applications, manages 12,000+ secrets, routes 47M+ inference requests per month, and delegates credentials to autonomous AI agents. A single vulnerability in any of these systems can compromise the entire platform.

Testing, no matter how thorough, cannot cover all possible states. Formal verification provides mathematical guarantees that critical properties hold for *all* inputs and *all* execution paths. We use Lean 4 [7] as our proof assistant because:

1. **Small trusted kernel**: Lean 4's type checker is the only trusted component. All proof steps are verified by the kernel.
2. **Dependent types**: Enable precise specifications that capture security invariants as types.
3. **Metaprogramming**: Lean 4's tactic framework enables automation of repetitive proof steps.
4. **Executable specifications**: Lean 4 code can be extracted to executable programs, enabling specification-implementation comparison.

1.1 Scope and Methodology

We verify properties at the specification level: we formalize the intended behavior of each system as Lean 4 definitions and prove theorems about those specifications. This catches design-level bugs (incorrect algorithms, missing edge cases, violated invariants) but does not verify the implementation code directly.

The verification process for each system follows four steps:

1. **Formalize**: Encode the system's data structures, state transitions, and external interfaces as Lean 4 types and functions.
2. **Specify**: State the desired properties as Lean 4 theorems.
3. **Prove**: Construct proofs accepted by the Lean 4 kernel.
4. **Audit**: Map proved properties back to implementation code and verify that the implementation matches the formalization.

2 Proof Suite Overview

System	Lean Lines	Theorems	Bugs Found
Agent Framework	3,200	47	3
API Gateway	2,800	38	2
KMS Secrets	2,100	31	1
Compute Scheduler	2,400	35	1
Platform Orchestrator	1,500	22	1
Total	**12,000**	**173**	**8**

Table 1: Formal verification suite summary. All 173 theorems are mechanically verified by the Lean 4 kernel (no `sorry`).

3 Agent Framework Verification

The Hanzo Agent SDK [1] enables AI agents to perform actions with delegated credentials. Three properties are critical:

3.1 Safe Delegation

An agent must never acquire permissions beyond what its delegator possesses.

Definition 1 (Permission Lattice)**.** *Permissions form a bounded lattice* $(\mathcal{P}, \sqsubseteq, \sqcup, \sqcap, \bot, \top)$ *where* $p_1 \sqsubseteq p_2$ *means "*p_1 *is a subset of* p_2*." The lattice operations:*

- $\sqcup$ *(join): Union of permissions.*
- $\sqcap$ *(meet): Intersection of permissions.*
- $\bot$*: No permissions.*
- $\top$*: All permissions (root only).*

Theorem 1 (Safe Delegation)**.** *For any delegation chain* $u \to a_1 \to \cdots \to a_k$ *where* u *is a user with permissions* p_u *and each* a_i *delegates to* a_{i+1} *with constraint* c_i*:*

$$p_{a_k} \sqsubseteq p_{a_{k-1}} \sqsubseteq \cdots \sqsubseteq p_{a_1} \sqsubseteq p_u. \tag{1}$$

That is, permissions monotonically decrease along any delegation chain.

The Lean 4 proof proceeds by induction on the chain length k, with the base case ($k = 1$) following from the delegation rule $p_{a_1} = p_u \sqcap c_1 \sqsubseteq p_u$.

3.2 Credential Chain Validity

Theorem 2 (Chain Validity)**.** *A credential chain is valid if and only if:*

1. *The root certificate is signed by the platform root key.*
2. *Each intermediate certificate is signed by its parent.*
3. *Each certificate's permissions are* $\sqsubseteq$ *its parent's.*

4. *No certificate has expired.*

5. *No certificate has been revoked.*

The verification algorithm terminates in $O(k)$ time for chain length k and returns `true` *if and only if all five conditions hold.*

3.3 Bug Found: Permission Escalation via Revocation Race

Formalization revealed a race condition: if a parent credential is revoked while a child agent is mid-operation, the agent could complete the operation with permissions derived from a now-revoked credential. The fix adds a real-time revocation check at operation execution time, not just at credential presentation time.

4 API Gateway Verification

The Hanzo Gateway [2] routes all API requests. We verify three properties.

4.1 Routing Correctness

Definition 2 (Route Table)**.** *A route table $\mathcal{T} : \texttt{Path} \times \texttt{Method} \to \texttt{Backend}$ maps (path, HTTP method) pairs to backend services. Routes have priorities; the highest-priority matching route is selected.*

Theorem 3 (Route Determinism)**.** *For any request (p, m), the routing function $\texttt{route}(\mathcal{T}, p, m)$ returns exactly one backend service or* `404`*. No request matches two routes with equal priority (this is enforced as a well-formedness condition on $\mathcal{T}$).*

4.2 Rate Limiter Fairness

Definition 3 (Token Bucket)**.** *A token bucket $B = (r, b, t)$ has refill rate r tokens per second, burst capacity b, and current tokens t. A request consuming c tokens is admitted if $t \geq c$.*

Theorem 4 (Rate Limiter Fairness)**.** *For two organizations with identical rate limit configurations (r, b), the maximum difference in admitted requests over any time window $[0, T]$ is bounded by the burst capacity b:*

$$|A_1(T) - A_2(T)| \leq b. \tag{2}$$

4.3 Authentication Chain Soundness

Theorem 5 (Auth Soundness)**.** *If the gateway admits a request with org claim o, then:*

1. *A valid JWT exists, signed by the IAM signing key.*

2. *The JWT has not expired.*

3. *The org o in the JWT matches the org injected into downstream headers.*

4. *All downstream data queries are scoped to org o.*

No combination of request headers can cause the gateway to inject an org different from the one in the validated JWT.

4.4 Bug Found: Path Traversal in Route Matching

Formalization revealed that URL-encoded path separators (`%2F`) were not normalized before route matching, allowing requests to bypass path-based access controls. The fix adds URL normalization as the first step in the routing pipeline.

5 KMS Secrets Verification

Hanzo KMS [3] manages 12,000+ secrets with envelope encryption.

5.1 Envelope Encryption Correctness

Definition 4 (Envelope Encryption)**.** *A secret s is encrypted as $\mathtt{Enc}_{KEK}(\mathtt{DEK}) \| \mathtt{Enc}_{DEK}(s)$ where DEK is a per-secret data encryption key and KEK is the master key encryption key.*

Theorem 6 (Encryption Correctness)**.** *For any secret s and keys (KEK, DEK):*

$$\mathtt{Dec}_{DEK}(\mathtt{Enc}_{DEK}(s)) = s, \tag{3}$$

and

$$\mathtt{Dec}_{KEK}(\mathtt{Enc}_{KEK}(DEK)) = DEK. \tag{4}$$

Composition: the full decrypt-unwrap pipeline recovers s exactly.

5.2 Key Rotation Safety

Theorem 7 (Rotation Safety)**.** *During key rotation from KEK_{old} to KEK_{new}:*

1. *No secret s is ever stored in plaintext (the DEK is always encrypted under at least one KEK).*
2. *No secret is inaccessible (either KEK_{old} or KEK_{new} can decrypt the DEK at every point during rotation).*
3. *After rotation completes, KEK_{old} can be safely destroyed without losing access to any secret.*

The proof models rotation as a state machine with five states (idle, re-wrap started, re-wrap in progress, re-wrap completed, old key destroyed) and proves that the safety invariant holds at every state transition.

5.3 Bug Found: Partial Rotation Failure

Formalization revealed that if the re-wrap process fails mid-way (e.g., database connection lost), some secrets remain wrapped under the old KEK while others are wrapped under the new KEK. If the old KEK is then destroyed (automated cleanup), the un-re-wrapped secrets become inaccessible. The fix adds a rotation completion check before old key destruction.

6 Compute Scheduler Verification

The Hanzo compute scheduler [4] allocates GPU resources to inference and training jobs.

6.1 Resource Allocation Fairness

Definition 5 (Weighted Fair Queuing)**.** *For n organizations with weights $w_1, \ldots, w_n$ and a pool of G GPUs, weighted fair allocation assigns:*

$$g_i = \left\lfloor G \cdot \frac{w_i}{\sum_j w_j} \right\rfloor \tag{5}$$

GPUs to organization i, with remaining GPUs assigned by weight-priority.

Theorem 8 (Allocation Fairness)**.** *The scheduler's allocation satisfies:*

$$\left| \frac{g_i}{G} - \frac{w_i}{\sum_j w_j} \right| < \frac{1}{G} \quad \forall i. \tag{6}$$

No organization receives more than $1/G$ deviation from its fair share.

6.2 Deadlock Freedom

Theorem 9 (Deadlock Freedom)**.** *The compute scheduler with priority-based preemption is deadlock-free: for any finite set of jobs, every job either completes or is explicitly rejected within bounded time. No circular wait condition can occur because:*

1. *Resources (GPUs) are preemptible by higher-priority jobs.*
2. *Jobs do not hold resources while waiting for other resources.*
3. *A total order on job priorities breaks symmetry.*

6.3 Bug Found: Priority Inversion

A high-priority realtime inference job could be blocked by a low-priority batch training job occupying the last available GPU, with medium-priority jobs preventing the batch job from completing. The fix implements priority inheritance: the batch job temporarily inherits the high-priority job's priority, ensuring it completes and releases the GPU.

7 Platform Orchestrator Verification

The platform orchestrator manages service deployments across Kubernetes clusters.

7.1 Deployment State Machine

Definition 6 (Deployment States)**.** *A deployment d transitions through states $\mathcal{S} = \{$`pending`, `building`, `deploying`
with well-defined transitions.

Theorem 10 (State Machine Correctness)**.** *The deployment state machine satisfies:*

1. ***Reachability**: Every non-terminal state can eventually reach `running` or `failed`.*
2. ***No stuck states**: No state has zero outgoing transitions (except terminal states).*
3. ***Rollback safety**: From `rolling_update`, the system can always transition to `rolling_back`, which always reaches the previous `running` configuration.*

7.2 Health Check Liveness

Theorem 11 (Health Check Liveness)**.** *If a service is healthy (responds to health probes within timeout), the orchestrator eventually marks it as* `running`*. If a service is unhealthy (fails k consecutive health probes), the orchestrator eventually restarts it. No healthy service is restarted, and no unhealthy service remains running indefinitely.*

7.3 Bug Found: Rolling Update with Zero Healthy Replicas

During a rolling update, if all new replicas fail health checks simultaneously, the orchestrator should roll back. However, the implementation allowed the update to proceed until `maxUnavailable` was reached, leaving the service with zero healthy replicas before triggering rollback. The fix adds a minimum healthy replica invariant: rollback triggers immediately if healthy replicas drop below the configured minimum.

8 Proof Engineering

8.1 Lean 4 Proof Architecture

The proof suite is organized into five Lean 4 packages, one per system, with a shared foundation package containing common definitions (permission lattice, cryptographic primitives, state machine framework).

8.2 Automation

Approximately 60% of proof steps are automated using Lean 4 tactics:

- `simp`: Simplification for algebraic properties.
- `omega`: Linear arithmetic over integers and naturals.
- `decide`: Decidable propositions over finite types.
- Custom tactics for state machine reachability and permission lattice reasoning.

8.3 Verification Time

Full verification (type-checking all 12,000 lines) completes in 47 seconds on a 16-core machine. This runs in CI on every commit to the proof repository.

9 Related Work

seL4 [8] verified an OS microkernel with 200,000 lines of Isabelle/HOL proofs. Our scope is narrower (infrastructure properties, not kernel correctness) but covers more systems.

CompCert [9] verified a C compiler in Coq. We verify at the specification level rather than implementation level, trading completeness for broader coverage.

IronFleet [10] verified distributed systems (Paxos, a sharded KV store) in Dafny. Our work addresses AI-specific properties (agent delegation, inference scheduling) not present in traditional distributed systems.

Lux Network [6] has its own formal verification suite for consensus properties. Our proofs complement the Lux proofs by verifying the application-layer infrastructure built on top.

10 Conclusion

The Hanzo formal verification suite provides mechanically-checked proofs of 173 critical properties across five infrastructure systems. The formalization effort discovered 8 bugs undetected by testing, including a permission escalation via revocation race, a path traversal in route matching, and a rolling update failure mode leaving zero healthy replicas. All 12,000 lines of Lean 4 proofs are checked by the Lean kernel with no `sorry` axioms, providing mathematical certainty that the specified properties hold. The proof suite runs in CI in 47 seconds, ensuring that specification changes are continuously verified.

References

[1] Hanzo AI Research. Hanzo Agent SDK: Framework for building AI agents. *Hanzo AI Research*, 2025.

[2] Hanzo AI Research. Hanzo Gateway: API gateway with KrakenD backend. *Hanzo AI Research*, 2024.

[3] Hanzo AI Research. Hanzo KMS: Secrets management with per-environment rotation and audit trail. *Hanzo AI Research*, September 2021.

[4] Hanzo AI Research. Hanzo ACI: AI Chain Infrastructure for decentralized compute markets. *Hanzo AI Research*, December 2024.

[5] Hanzo AI Research. Hanzo IAM: Identity and access management for AI platforms. *Hanzo AI Research*, 2024.

[6] Lux Partners. Lux: A scalable multi-consensus blockchain with post-quantum cryptography. *Lux Network Technical Report*, 2024.

[7] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Proc. CADE*, 2021.

[8] G. Klein, K. Elphinstone, G. Heiser, et al. seL4: Formal verification of an OS kernel. In *Proc. SOSP*, 2009.

[9] X. Leroy. Formal verification of a realistic compiler. *Commun. ACM*, 52(7):107–115, 2009.

[10] C. Hawblitzel, J. Howell, M. Kapritsos, J. R. Lorch, B. Parno, M. L. Roberts, S. Setty, and B. Zill. IronFleet: Proving practical distributed systems correct. In *Proc. SOSP*, 2015.